As `IOptionsSnapshot<AppDisplaySettings>` is registered as a scoped service, it's re- created at every request. If you edit the settings file and cause `IConfiguration` to reload, `IOptionsSnapshot<AppDisplaySettings>` shows the new values on the next request. A new `AppDisplaySettings` object is created with the new configuration values and is used for all future DI—until you edit the file again, of course!

Reloading your settings automatically is as simple as that: update your code to use `IOptionsSnapshot<T>` instead of `IOptions<T>` wherever you need it. But be aware that this change isn't free. You're rebinding and reconfiguring your options object with every request, which may have performance implications. In practice, reloading settings isn't common in production, so you may decide that the developer convenience isn't worth the performance impact.

An important consideration in using the options pattern is the design of your POCO options classes themselves. These classes typically are simple collections of properties, but there are a few things to bear in mind so that you don't get stuck debugging why the binding seemingly hasn't worked.

10.3.3 Designing your options classes for automatic binding

I've already touched on some of the requirements for POCO classes to work with the `IOptions<T>` binder, but there are a few rules to remember. The first key point is that the binder creates instances of your options classes by using reflection, so your POCO options classes need to

- Be nonabstract
- Have a default (`public` parameterless) constructor